

Binary indexed tree (BIT) is a data structure that stores an array of values, and can efficiently compute prefix sums of the values and update the values.

The problem:

Say we have an array

Original array: $x_1, x_2, \dots, x_n$ (where n could be super large)

We might often want to know $z_k = x_1 + \dots + x_k$, i.e. prefix sum

To get z_k , the computational complexity is $O(n)$.

So we might consider to store the prefix sum array directly.

Prefix sum array: $z_1, z_2, \dots, z_n$

Now to get z_k , the computational complexity is $O(1)$.

However, if x_k needs to be updated, we have to update $z_k, \dots, z_n$ (computational complexity $O(n)$)

BIT:

Binary indexed tree strikes a balance between search and updates.

To store $x_1, x_2, \dots, x_n \xrightarrow{\text{build}} y_1, \dots, y_n$ (binary indexed tree)

BIT let $y_i = x_i$ if i is odd

$y_i = x_i + x_{i-1}$ if i is even but $\nexists k$ such that $2^k = i$

$y_i = x_1 + \dots + x_i$ if $\exists k$ such that $2^k = i$

Notes:

1. Construct a BIT is $O(\log n)$

Say we want to add a new value x_{n+1} into BIT.

If $n+1$ is odd $\leadsto y_{n+1} = x_{n+1} \quad (O(1))$

If $n+1$ is even but $\nexists k: 2^k = n+1$, then $y_{n+1} = x_{n+1} + x_n \quad (O(1))$

If $\exists k: 2^k = n+1$, then $y_{n+1} = x_{n+1} + x_n + y_{n-1} + y_{n-3} + \dots + y_{(n+1)/2} \quad (O(\log(n)))$

2. Search in BIT is $O(\log n)$

Say we want to know $z_i = x_1 + \dots + x_i$

if $\exists k: 2^k = i$, then $z_i = y_i \quad (O(1))$

if i is even, but $\nexists k: 2^k = i$, then $z_i = y_i + y_{i-2} + \dots + y_j$

if i is odd, then $z_i = y_i + y_{i-1} + y_{i-3} + \dots + y_j$ $\xrightarrow{\text{max}\{j: 2^k = j, j < i\}}$

3. update in BIT is $O(\log n)$

if we want to update x_i

then if i is odd $\leadsto$ update $y_i, y_{i+1}, \dots, y_{j_k} \quad (O(\log n))$

if i is even and $\exists k: 2^k = i$,

update

if i is even but $\nexists k: 2^k = i$

update

$j_1, j_2, \dots, j_k$ such that

$\exists k: 2^k = j$ and

$j < n$

implementation trick:

in order to build, search and update the BIT, we need to jump between index. E.g. To get z_7 , we need y_7 , y_6 and y_4 .

We can find indices 7, 6, 4 by coding some rules, but there is a trick to use lowest set bit (LSB)

note that $7 \xrightarrow{\text{binary}} 00000111$ ↗ say we have 8 bits

$6 \xrightarrow{\text{binary}} 00000110$ ↘

$4 \xrightarrow{\text{binary}} 00000100$ ↘

$0 \xrightarrow{\text{binary}} 00000000$ ↘

We are removing the last 1 at each step.

in order to get last 1, we use the trick $\text{index} \& (-\text{index})$

$$7 \rightsquigarrow 00000111$$

$$-7 \rightsquigarrow 11111000 + 1 \rightsquigarrow 11111001$$

$$\rightarrow 7 \& -7 \rightsquigarrow 00000001 \quad (\text{the last 1 in 7})$$

$$6 \rightsquigarrow 00000110$$

$$-6 \rightsquigarrow 11111001 + 1 \rightsquigarrow 11111010$$

$$\rightarrow 6 \& -6 \rightsquigarrow 00000010 \quad (\text{the last 1 in 6})$$